

# Balance Criteria를 사용하지 않는 간단한 균형이진탐색트리

김태우<sup>○</sup>

전산학부, 한국과학기술원  
travis1829@kaist.ac.kr

## A Simple Balanced Binary Search Tree with No Balance Criteria

Tae Woo Kim<sup>○</sup>

School of Computing, KAIST

### 요 약

일반적으로 균형이진탐색트리는 자기만의 특수한 balance criteria를 통해 트리의 균형이 깨졌는지를 판단한 후 필요한 rebalancing을 수행하여 트리의 높이를 적절하게 유지한다. 하지만 이것이 균형이진탐색트리의 구현을 복잡하게 하는 것으로 악명높기에, 본 논문에서는 balance criteria를 아예 사용하지 않음으로써 균형이진탐색트리를 쉽고 직관적으로 구현할 수 있는 방법을 소개하고자 한다. 이 방법을 구현하기 위해서는 평범한 이진탐색트리의 구현 코드에 간단한 코드만을 추가하면 되며, 탐색은  $O(\log n)$ , 삽입/삭제는 amortized  $O(\log n)$ 만에 수행할 수 있고, 간단한 방법을 통한 병렬화로 성능을 향상할 수도 있다.

### 1. 서론

균형이진탐색트리는 매우 기본적이고 중요한 자료구조이다. 평범한 이진탐색트리와 달리, 균형이진탐색트리는 자기만의 특수한 balance criteria 및 rebalancing 알고리즘을 이용하여, 수많은 key를 크기 순서대로 보관하면서 key의 탐색, 삽입, 삭제 등을  $O(\log n)$  또는 amortized  $O(\log n)$ 만에 수행할 수 있다.

그러나, 이 balance criteria와 rebalancing을 구현하는 것 때문에 균형이진탐색트리는 예로부터 구현이 어렵고 실수하기 쉽기로 악명 높았다. Red-Black Tree[1] 등의 일반적인 균형이진탐색트리들은 트리의 높이를  $O(\log n)$ 으로 유지하기 위해 트리만의 특수한 balance criteria를 준수하도록 트리의 구조를 제한하고, 트리가 이를 어기게 되면 tree rotation 또는 partial rebuild<sup>(1)</sup>을 이용한 rebalancing을 통해 다시 criteria를 지키는 구조가 되도록 바꾼다. 하지만, Andersson[2]에 의하면 이러한 criteria들이 너무 복잡하며, Munro et al.[3]에 의하면 이러한 rotation 등을 이용한 rebalancing을 구현하기 위해서는 트리의 여러 경우를 모두 따져보면서 트리의 구조를 복잡한 과정을 통해 변경해야 하기에 평범한 프로그래머 입장에서는 구현이 너무 복잡해진다. 그렇기에, 관련 연구들[2,4]은 주로 더 간단한 criteria로도 트리의 높이를  $O(\log n)$ 으로 유지할 수 있는 방법을 찾아보곤 하였다.

이 문제는 균형이진탐색트리를 목적에 맞게 변형해야 하는 경우가 잦기에 더 심각해진다. 예로, 트리의  $k$ th element, 구간에 속하는 key의 수 등을  $O(\log n)$ 만에 찾는 것은 추가 메모리가 필요하므로 표준 라이브러리가 지원하지 않는 경우가 많다.

또, 최근의 Stack Overflow 설문조사[5] 등에 의하면 프로그래밍을 하는 사람들의 경력, 학력, 직업 등이 매우 다양하므로, 간단한 알고리즘에 대한 수요는 항상 컸다고 할 수 있다.

그러므로, 본 논문은 트리의 criteria를 간략화하는 것을 넘어, 아예 criteria를 사용하지 않으면서 키의 탐색은  $O(\log n)$ , 삽입/삭제는 amortized  $O(\log n)$ 만에 수행할 수 있는, 간단하면서도 직관적인 알고리즘을 소개하고자 한다.

(1) 기존의 unbalanced subtree로부터 perfectly balanced subtree를 만든 후, 이걸로 기존의 subtree를 대체하는 방법

### 2. 트리 알고리즘

이 논문에서 소개하는 트리는 트리가 criteria를 어길 때마다 rebalancing을 하지 않고, 그 대신 subtree마다 timer를 통해 rebuild를 언제 할지를 예약해둔다. 자세히는 다음과 같다.

- 1) 모든 subtree는 정수 timer를 가지고 있다.
- 2) subtree에 노드를 삽입 또는 삭제할 때마다 그 subtree의 timer가 1 감소한다.
- 3) subtree의 timer가 0이 되면, subtree를 rebuild하고 timer를  $\max(1, \lfloor kn \rfloor)$ 으로 재설정한다.  $k$ 는  $(0, 1)$ 에 속하는 미리 정해진 상수이고,  $n$ 은 subtree의 크기<sup>(2)</sup>이다.
- 4) subtree의 timer는 subtree의 root 노드에 저장하며, 새롭게 삽입된 노드의 timer는 1부터 시작한다.

이 방법을 사용하는 경우, 탐색 operation은 평범한 이진탐색트리에서와 똑같이 수행하면 되며, 삽입/삭제 operation은 다음과 같이 수행하면 된다.

#### ■ 삽입

- 1) 평범한 이진탐색트리에서와 같이, 재귀 알고리즘을 이용하여 leaf 노드를 삽입한다.
- 2) 새로운 leaf 노드부터 root 노드까지의 경로를 재방문하면서, 모든 노드  $t$ 에 대해 Update( $t$ )를 한다.

#### ■ 삭제

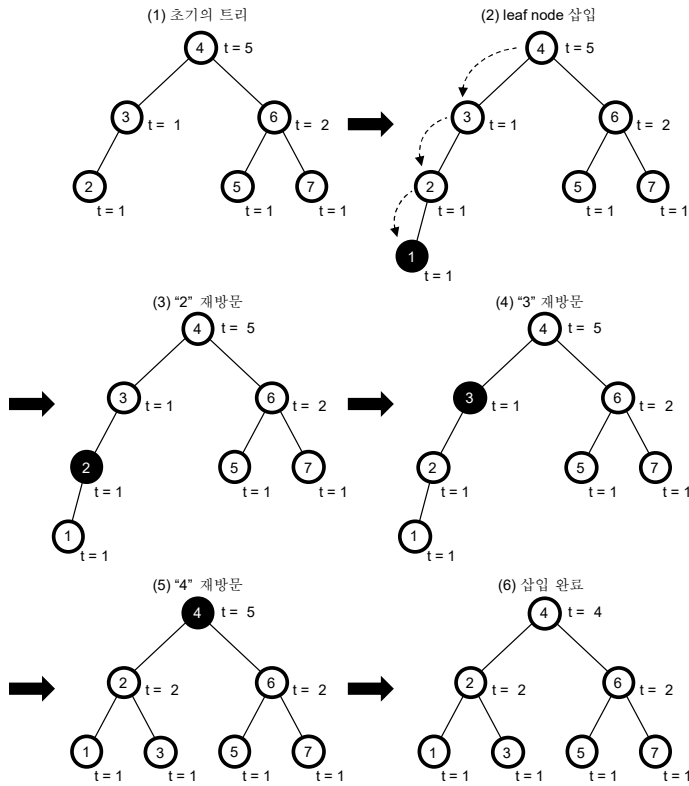
- 1) 평범한 이진탐색트리에서와 같이, 재귀 알고리즘을 이용하여 leaf 노드<sup>(3)</sup>를 삭제한다.
- 2) 삭제된 leaf 노드부터 root 노드까지의 경로를 재방문하면서, 모든 노드  $t$ 에 대해 Update( $t$ )를 한다.

#### ■ Update( $t$ )

- 1)  $t$ 의 timer를 1 감소시킨다.
- 2)  $t$ 의 timer가 0이면  $t$ 를 root로 갖는 subtree를 rebuild한다.

(2) subtree의 크기는 subtree의 노드 개수로 정의한다.

(3) 만약 삭제하려는 노드가 leaf 노드가 아니라면, 이 문제를 leaf 노드를 삭제하는 문제로 변환시키는 일반적인 방법[6]을 사용한다.



[그림 1] 트리에 "1"을 삽입하는 과정의 예  
( $k = 0.9$ 이며, timer는  $t$ 로 축약하였음)

#### ■ Rebuild

- 1) 중위 탐색(inorder traversal)을 이용하여 subtree의 모든 노드들을 1차원 배열로 순서대로 복사한다.
- 2) 분할 정복을 이용하여 배열로부터 perfectly balanced subtree를 만들며, 이 과정에서 모든 노드의 timer도 재설정한다. 이 subtree로 원래 subtree를 대체한다.

다음은 Update( $t$ ), Rebuild에 해당하는 부분의 의사 코드(pseudo code)이다.

Update( $t$ )

```
t.timer ← t.timer - 1
if t.timer = 0 then
    arr ← empty array
    GetCopy(t, arr)
    t ← BuildTree(arr, 0, arr.size - 1)
```

GetCopy( $t$ , arr)

```
if t ≠ null then
    GetCopy(t.left, arr)
    arr.pushBack(t)
    GetCopy(t.right, arr)
```

BuildTree(arr, s, f)

```
if s > f then
    return null
m ← ⌊(s + f)/2⌋
arr[m].left ← BuildTree(arr, s, m - 1)
arr[m].right ← BuildTree(arr, m + 1, f)
arr[m].timer ← max(1, ⌊k · (f - s + 1)⌋)
return arr[m]
```

더 자세한 C++, Python 구현 예시[7]에서 확인할 수 있다.

이 방법을 사용할 경우, 구현의 난이도에 있어 다음과 같은 이점들이 있다.

- Balance criteria가 없으므로, 트리를 rebalance할지 말지를 매우 간단하게 결정할 수 있다.
- 삽입과 삭제에 사용되는 rebalancing이 같으며, 평범한 이진탐색트리의 삽입/삭제 operation을 재귀 알고리즘으로 구현한 코드의 마지막 부분에 Update( $t$ )만 추가하면 된다 (위 구현 예에서는 삽입/삭제 각각에 한 줄만 추가).
- 평범한 이진탐색트리와 비교하여 추가로 필요한 코드의 경우, Update( $t$ )는 간단하며, Rebuild 또한 트리 순회 알고리즘과 간단한 분할 정복 알고리즘과 같이 기본적인 알고리즘만으로 구현할 수 있다.
- 트리의 현재 구조를 정확히 파악해야 하는 tree rotation에 비해, partial rebuild는 고려해야 할 사항이 적으므로, 구현 과정에서 일어나는 실수를 줄일 수 있다.

#### 3. Rebuild의 병렬화

추가로, 많은 균형이진탐색트리들이 수행 시간의 상당 부분을 rebalancing하는 데 쓰게 되는데, tree rotation과 다르게 rebuild는 race 등에 대한 걱정 없이 쉽게 병렬화가 가능하다.

본 논문의 트리를 예로 들면, 간단한 변경을 통해 모든 subtree가 자신의 크기를 저장하도록 한다면, GetCopy가 arr.pushBack을 사용할 필요 없이 arr의 미리 계산된 인덱스에 직접 노드를 복사하도록 수정할 수 있다. 이후, 필요에 따라, 재귀함수 GetCopy와 BuildTree가 재귀호출을 하는 대신 그 함수를 호출하는 새로운 thread를 만들도록 변경한다면, rebuild 과정이 멀티 thread를 활용하도록 쉽게 변경할 수 있다.

#### 4. 트리의 높이 분석

본 논문의 알고리즘에서, 노드는 처음 삽입되었을 때 timer가 1로 설정되고, 노드를 root로 가지는 subtree가 rebuild될 때는 timer가 재설정된다. 즉, 모든 노드는 마지막으로 timer가 설정 또는 재설정됐던 때가 항상 존재한다. 이로부터, 다음을 확인할 수 있다.

##### 보조정리 1

모든 subtree는 다음의 두 가지 중 하나의 경우에 속한다.

- 1) timer가 마지막으로 (재)설정되었던 때로부터, 이 subtree에 추가적인 삽입 또는 삭제는 이루어지지 않았다.
- 2) timer가 마지막으로 (재)설정되었던 때로부터, 이 subtree에 추가적인 삽입 또는 삭제가 적어도 한번 이루어졌다.

이후, 다음을 확인할 수 있다.

##### 보조정리 2

크기  $n$ 의 perfectly balanced subtree의 모든 proper subtree의 크기는  $\frac{n}{2}$  이하이다.

##### 정리 1

timer가 (재)설정된 subtree  $T$ 에 최대  $x$ 개의 삽입 또는 삭제를 할 수 있다고 하자. 모든 경우에 대해,  $T$ 의 크기가 항상  $n$ 이하라면,  $T$ 의 proper subtree의 크기 또한 항상  $\frac{1+2k}{2+2k}n$  이하이다.

(증명) 귀류법(proof by contradiction)을 이용한다.

즉,  $T$ 의 timer가 마지막으로 (재)설정되었을 때의  $T$ 의 크기가  $n_0$ 였고, 여기에  $u$ 개의 삽입 및 삭제를 하였더니  $T$ 의 proper

subtree의 크기  $m$ 이  $\frac{1+2k}{2+2k}n$ 를 초과하였다고 가정하자. 그러

면, 보조정리 2에 의하여  $m \leq \frac{n_0}{2} + u$ 이므로,  $\frac{1+2k}{2+2k}n < \frac{n_0}{2} + u$ 이다. 하지만  $0 \leq u < \max(1, \lfloor kn_0 \rfloor)$ 이고, 가정에 의해  $n \geq n_0 + u$ 이므로, 이는 다음에 의해 모순이다.

1)  $n_0 < \frac{2}{k}$ 인 경우  $u=0$ 이므로,

$$\frac{n_0}{2} + u = \frac{n_0}{2} \leq \frac{1+2k}{2+2k}n_0 \leq \frac{1+2k}{2+2k}n.$$

2)  $n_0 \geq \frac{2}{k}$ 인 경우,  $u \leq \lfloor kn_0 \rfloor - 1 < kn_0$ 이다.

하지만  $\frac{1+2k}{2+2k}n < \frac{n_0}{2} + u$ 이라 가정할 경우,

$$\frac{1+2k}{2+2k}(n_0 + u) \leq \frac{1+2k}{2+2k}n < \frac{n_0}{2} + u \text{이므로, } u > kn_0 \text{이다.}$$

즉, 모순이므로, 불가능하다.  $\square$

## 정리 2

트리의 높이는  $\left\lceil \log_{\frac{2+2k}{1+2k}} \frac{1+2k}{2-2k} n \right\rceil$  이하이다. 즉, 탐색 operation의 비용은  $O(\log n)$ 이다.

(증명)

1) 트리가 보조정리 1의 경우 1)에 속하는 경우, perfectly balanced이다. 즉, 높이는  $\lfloor \log_2 n \rfloor$  이하이다.

2) 트리가 보조정리 1의 경우 2)에 속한다고 가정하자. 그리고, 트리의 timer가 마지막으로 (재)설정되었을 때의 크기를  $n_0$ 라 한다면, 트리가 이 이후에 수행한 삽입 또는 삭제의 수는

$\lfloor kn_0 \rfloor - 1$ 이하이며, 경우 2)에 속하기 위해서는  $n_0 \geq \frac{2}{k}$ 이 필요하다. 그러므로, 트리의 왼쪽 또는 오른쪽 subtree의 크기는 어떤 경우에도  $\frac{n_0}{2} + \lfloor kn_0 \rfloor - 1$  이하이고, 트리의 현재 크기  $n$

은  $n_0 - \lfloor kn_0 \rfloor + 1$  이상이며, 이 둘을 나눈 몫은  $\frac{1+2k}{2-2k}$  미만이다. 그러므로, 정리 1에 의해, depth  $d$ 의 노드를 root로 가지는 subtree의 크기는 항상  $\left(\frac{1+2k}{2+2k}\right)^{d-1} \left(\frac{1+2k}{2-2k}\right)n$  미만이다.

그러므로, 트리의 높이는  $\left\lceil \log_{\frac{2+2k}{1+2k}} \frac{1+2k}{2-2k} n \right\rceil$  이하이다.  $\square$

## 5. 삽입/삭제 operation의 비용 분석

### 정리 3

크기  $n$ 의 subtree  $T$ 를 rebuild하는 비용이  $n$ 이라 하자. 그러면, subtree  $T$ 의 rebuild 비용을 subtree가 마지막 timer (재)설정 이후에 했던 삽입 및 삭제의 수  $u$ 로 나눈 몫은  $\frac{2}{k} + 1$  미만이다.

(증명)  $T$ 의 timer가 마지막으로 (재)설정되었을 때의  $T$ 의 크기를  $n_0$ 라 하자. 그러면,  $u = \max(1, \lfloor kn_0 \rfloor)$ 이고  $n \leq n_0 + u$ 임을 이용하여 다음을 확인할 수 있다.

1)  $n_0 < \frac{1}{k}$ 이면,  $u=1$ 이므로,  $\frac{n}{u} < \frac{1}{k} + 1$ 이다.

2)  $n_0 \geq \frac{1}{k}$ 이면,  $u = \lfloor kn_0 \rfloor$  이므로,  $\frac{n}{u} \leq \frac{n_0}{\lfloor kn_0 \rfloor} + 1$ 이다.

이때,  $n_0$ 는 항상 어떤 자연수  $a \geq 1$ 에 대해  $\frac{a}{k} \leq n_0 < \frac{a+1}{k}$

이므로,  $\frac{n_0}{\lfloor kn_0 \rfloor} + 1 = \frac{n_0}{a} + 1 < \left(\frac{a+1}{a}\right) \frac{1}{k} + 1 \leq \frac{2}{k} + 1$ 이다.

즉, 몫은 항상  $\frac{2}{k} + 1$  미만이다.  $\square$

그러므로, 삽입 또는 삭제를 할 때마다 삽입/삭제된 노드의

모든 ancestor 노드에  $\frac{2}{k} + 1$ 의 credit을 추가로 적립하면,

subtree가 rebuild될 때 그 subtree의 root 노드는 항상 rebuild 비용을 낼 수 있을 만큼의 충분한 credit을 갖고 있게 된다. 삽입/삭제된 노드의 ancestor 노드의 수는  $O(\log n)$ 이므로, 분할 상한분석(amortized analysis)을 통해 다음을 확인할 수 있다.

### 정리 4

삽입/삭제 operation의 비용은 amortized  $O(\log n)$ 이다.

## 6. Scapegoat tree와의 비교

추가로, 이 트리는 partial rebuild를 쓴다는 점에서 Scapegoat tree[8](또는 General balanced tree[4])와 비슷하지만, 다음과 같은 차이점이 있다.

먼저, 트리의 구조가 서로 다르다. 예로, Scapegoat tree는 subtree의 크기로 그 subtree의 왼쪽/오른쪽 subtree의 크기를 나눈 몫이 항상 특정 범위 내에 있도록 제한하지만, 이 트리는 구조가 훨씬 유연하여 몫을 특정 범위 내로 제한할 수 없다.

또한, 이 트리는 balance criteria가 없고, rebuild를 할 위치도 매우 쉽게 찾을 수 있다. 그러나, Scapegoat tree는 insert와 delete가 서로 다른 criteria를 사용해야 하며, 또, insert의 경우, rebuild를 할 위치를 찾기 위해선 노드를 삽입한 직후 leaf부터 root까지의 노드들을 순차적으로 재방문하며 각각의 노드의 모든 자손 노드를 재방문해야 하는 등, 병렬화가 곤란한 복잡한 과정을 통해 rebuild를 할 위치를 찾아야 한다.

## 7. 결론

Balance criteria를 단순화하는 것을 넘어, 아예 balance criteria를 사용하지 않는 알고리즘을 사용함으로써 균형이진 탐색트리를 더 쉽게 구현할 수 있음을 확인하였다. 특히, 이진 탐색트리의 코드에 간단한 코드만을 추가함으로써 구현할 수 있으므로, 초보자의 경우, 주로 탐색 operation만을 사용하는 경우, 또는 amortized  $O(\log n)$ 의 성능이면 충분한 상황 등에서 적합한 방법으로 사용될 수 있을 것이다.

## 참고 문헌

- [1] L. J. Guibas and R. Sedgewick, "A dichromatic framework for balanced trees," Proceedings of the 19th Annual Symposium on Foundations of Computer Science, pp. 8-21, 1978.
- [2] A. Andersson, "Balanced Search Trees Made Simple," Proceedings of the Third Workshop on Algorithms and Data Structures, pp. 60-71, 1993.
- [3] J. I. Munro, T. Papadakis, and R. Sedgewick, "Deterministic Skip Lists," Proceedings of the Third Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 367-375, 1992.
- [4] A. Andersson, "General Balanced Trees," Journal of Algorithms, pp. 1-18, 1999.
- [5] "Stack Overflow Developer Survey 2020," 2020. [Online]. Available : <https://insights.stackoverflow.com/survey/2020>. [Accessed Oct. 16, 2020].
- [6] R. Sedgewick, K. Wayne, Algorithms Fourth Edition, Pearson Education, p.410, 2011.
- [7] "travis1829/A-simple-balanced-search-tree," 2020. [Online]. Available : <https://github.com/travis1829/A-simple-balanced-search-tree>. [Accessed Oct. 16, 2020].
- [8] I. Galperin and R. L. Rivest, "Scapegoat Trees," Proceedings of the fourth annual ACM-SIAM Symposium on Discrete algorithms, pp. 165-174, 1993.